# The path policy lives here

*Animated explainer · src/workspace.js*

Every file path is resolved before the filesystem is touched. Write paths must stay inside the workspace. Read paths may leave it. The difference is the whole security story of this module.

**8 scenes · 2 resolve paths · 1 containment rule · symlink-aware**

CONTENTS

# What is in this explainer


01 · Why this file exists ..... 3

02 · Two paths ..... 4

03 · Write checks ..... 5

04 · Expansion ..... 6

05 · Symlinks ..... 7

06 · Containment ..... 8

07 · Reads ..... 9

08 · Recap ..... 10


Page numbers refer to the folio at the foot of each page.

# 01 · Why this file exists

*Resolve the path. Then touch the disk.*

**SRC/WORKSPACE.JS** · *source*

```
export class Workspace {
  constructor(root) {
    const abs = path.resolve(root);
    fs.mkdirSync(abs, { recursive: true });
    this.root = fs.realpathSync(abs);
  }

  resolve(requested) { /* writes: must stay inside */ }
  resolveRead(requested) { /* reads: may leave */ }
}
```

**resolve()** — Writes. Containment required.

**resolveRead()** — Reads. Open by design.

> **remember** The fence is around mutations, not curiosity. workspace.js is the path policy behind that sentence.
> **On screen:** Every path is resolved by this class before the filesystem is touched. There are two entry points: one for writes, one for reads. The constructor also resolves the real root, so the rest of the program shares one canonical location.

# 02 · Two paths

*Write inside. Read anywhere.*

- ✍️ resolve() expands variables, rejects escapes, and realpaths the target.
- 📖 resolveRead() allows absolute paths and returns a relative label that may start with ..
- 🧭 Shared rule relative paths start at the workspace root.

**SRC/WORKSPACE.JS** · *source*

```
resolveRead(requested) {
  const raw = requested.trim();
  const expanded = expandVars(raw);
  const abs = path.isAbsolute(expanded)
    ? path.resolve(expanded)
    : path.resolve(this.root, expanded);
  const real = realResolve(abs);
  const rel = path.relative(this.root, real) || '.';
  return { abs: real, rel: rel.split(path.sep).join('/') };
}
```

> **On screen:** Writes must stay inside the workspace. Reads are deliberately open. Both paths expand environment variables and resolve relative paths against the root. Reads return a relative label that may start with .., which is how the console can show an outside path.

# 03 · Write checks

*The obvious escapes are named.*

- 💽 Drive paths C:... is outside the workspace.
- 🖧 UNC paths \server\share is outside.
- 📁 Absolute paths refused for writes.
- 🏠 Home/system prefixes ~, $HOME, %USERPROFILE%, Desktop, Windows...
- 🔗 Symlinks realpath resolves the deepest existing ancestor.

**SRC/WORKSPACE.JS** · *source*

```
if (/^[A-Za-z]:/.test(raw)) throw new PathError('absolute or drive-relative path is outside the workspace');
if (/^(\\\\|\/\/)/.test(raw)) throw new PathError('UNC path is outside the workspace');
if (path.isAbsolute(expanded)) throw new PathError('absolute path is outside the workspace');

const first = raw.split(/[\\/]/)[0].toLowerCase().replace(/:$/, '');
if (/^[~$%]/.test(first) && first !== raw) throw new PathError('home/environment path is outside the workspace');
if (HOME_AND_SYSTEM.has(first)) throw new PathError('"' + first + '" looks like a home/system folder');
```

> **On screen:** A write path is checked for drive letters, UNC paths and absolute paths first. Then home and system-looking first segments are refused, because Windows may quietly resolve them inside the workspace. Only then does the path get joined to the root and realpath-checked.

# 04 · Expansion

*Expand variables before trusting a path.*

**SRC/WORKSPACE.JS** · *source*

```
function expandVars(p) {
  let out = p;
  if (out.startsWith('~')) out = path.join(os.homedir(), out.slice(1));
  out = out.replace(/%([^%]+)%/g, (m, name) => process.env[name] ?? m);
  out = out.replace(/\$\{?([A-Za-z_][A-Za-z0-9_]*)\}?/g, (m, name) => process.env[name] ?? m);
  return out;
}
```

The expansion is what makes ~/notes.txt a home path, not a workspace file called ~. The write check sees the real target.

**On screen:** expandVars turns ~, %NAME% and $NAME into the paths the operating system would use. That matters because a literal ~ may look harmless until the OS resolves it. Expansion happens before containment, so the check sees the real target.

# 05 · Symlinks

*A symlink must not smuggle a write out.*

**SRC/WORKSPACE.JS** · *source*

```
function realResolve(abs) {
  let cur = abs;
  const rest = [];
  while (!fs.existsSync(cur)) {
    const parent = path.dirname(cur);
    if (parent === cur) return abs;
    rest.unshift(path.basename(cur));
    cur = parent;
  }
  return path.join(fs.realpathSync(cur), ...rest);
}
```

A path that does not exist yet cannot be realpathed as a whole. This resolves the deepest existing ancestor, then appends the remainder.

**THE IDEA** · *text*

```
workspace/link -> C:\Users\me\
workspace/link/notes.txt  ->  C:\Users\me\notes.txt  ->  outside
```

**On screen:** Checking strings is not enough. A symlink or junction can point outside the root. realResolve finds the deepest part of the path that already exists and realpaths that. The remainder is appended, so a not-yet-created file under a symlinked folder is still judged by its real location.

# 06 · Containment

*One containment rule. No prefix tricks.*

**SRC/WORKSPACE.JS** · *source*

```
function isInside(root, target) {
  const rel = path.relative(root, target);
  return rel === '' || (!rel.startsWith('..') && !path.isAbsolute(rel));
}
```

A string prefix check would accept workspace-evil for a workspace root. path.relative does not.

**remember** Containment is a relationship between resolved paths, not a substring match.

**On screen:** isInside uses path.relative, not startsWith. The relative path must not begin with .. and must not be absolute. That one rule avoids the classic workspace-evil prefix bug.

# 07 · Reads

*A read outside is a bad idea, not an incident.*

- 📚 Docs and logs an agent that cannot read them is useless.
- 🌍 Absolute paths reads may leave the workspace.
- 📄 Bounded damage a read can only put text into the context window.
- 🔒 Writes stay fenced the mutation boundary is unchanged.

> **remember** The sandbox is around changes, not curiosity.
> **On screen:** Reads are not fenced. The agent may need the docs, a library or a log outside its folder. The damage a read can do is bounded by what enters the context window. Writes remain the line. That asymmetry is the design.

# 08 · Recap

*A small class with a large job.*

• 2 resolve paths

• 1 containment rule

• 0 raw path writes

→

**Resolve first.** No filesystem call before the path policy.

→

**Writes stay inside.** Drive, UNC, absolute and home paths are refused.

→

**Realpath deeply.** Symlinks do not get to smuggle a write.

→

**Reads stay open.** The fence is around mutation.

> **remember** A workspace is not a folder. It is a folder plus a policy for every path that touches it.
> **On screen:** Two resolve paths, one containment rule, no raw path writes. Resolve first, keep writes inside, realpath the ancestors, and leave reads open. That is the path policy behind the sandbox.